

SINGLE SOURCE OF TRUTH · MASTER ARCHITECTURE

SOURCE-A

AI Operating System

Master Architecture Document & SSOT

Version 1.2 — Production Locked & Agreed

Authoritative architecture framework for a multi-tenant factory runtime — now formalized around the Controlled Runtime Machine and the Execution Contract layer.

Plan → Control → Execute → Verify → Commit

Neon + Supabase Hybrid

Deterministic Kernel

What changed in v1.2

v1.0 established a strong decoupled structure, but it left the final control chain and production hardening unspecified. This release closes that gap: the kernel now holds absolute execution authority over the models, and the data tier moves to a hybrid engine.

5-Stage Control Loop

Every computational step now runs through a strict **Plan → Control → Execute → Verify → Commit** pipeline. The LLM plans; it never executes.

Execution Contract Layer

A new strict contract governs plan shape, allowed tools, expected output, and validation rules — the Source-A analogue to Cursor/Devin repo binding.

Hybrid Database

Neon Serverless Postgres becomes the primary State Engine (L3); Supabase is retained for auth, RLS, and realtime.

Refreshed Model Mix

Updated 2026 routing across Gemini 3.1 Flash-Lite, DeepSeek V4, Claude Sonnet, and Grok 4.1 Fast — capability-mapped, never hardcoded.

Architecture

Pillars, layers, and the controlled runtime machine

1 Architectural Philosophy — The 4 Pillars

The core tenet of the Source-A Operating System is that **specialization occurs in the configuration layer** (Data Graph / Blueprints), **not in the infrastructure layer** (Workers / Code).

Pillar	Name	Responsibility
P1	Product Layer UI / Config	Business skins and front-end interfaces (Forge, Noetfield, TrustField, WitnessBC, 777). Zero custom infrastructure — only presentation and specific configs.
P2	Governance Layer Policy / SLA	Quality-control logic, validation loops, compliance checks, and SLA definitions. Defines the unyielding rules of system execution.
P3	Runtime Kernel Stateless Workers	Completely stateless, memoryless Cloudflare Workers. Pure blind execution muscle, taking structured commands directly from data.
P4	Factory Data Graph Supabase / Neon	The connected multi-tenant ecosystem, orchestrated entirely via the Data Graph.

⚠ The Golden Rule

Workers are just muscle, never the brain. The brain of the system is the Data Graph stored in the persistent state engine. **LLMs do not execute directly; they only suggest steps. The Kernel maintains absolute Execution Authority.**

LLM $\neq$ Executor

LLM = Planner / Suggestion Engine

System = Execution Authority

2 The 8-Layer Kernel Architecture

Source-A is built around an unbundled, decoupled 8-layer stack for extreme cost efficiency, safety, and horizontal scale.

Layer	Component	Technology	Primary Role
L1	UI / Interface	Next.js + Cloudflare Pages	Unified multi-tenant dashboard scoped via <code>project_id</code> .
L2	Auth & Isolation	Supabase Auth + Postgres RLS	Multi-tenant cryptographic isolation and security enforcement.
L3	State Engine	Neon Serverless Postgres	High-write state tracking, live DAG state, branching, memory, and outputs.
L4	Queue Engine	Cloudflare Queues	Non-blocking async task distribution, throttling, and scheduling.
L5	Runtime Engine	4 Core Stateless Workers	Intake, Planner, Executor, and Memory worker processes.
L6	Capability Router	Dynamic Tool Registry	Capability-to-model abstraction based on SLA requirements.
L7	Heavy Compute	Modal / RunPod	Ephemeral heavy ops (FFmpeg, video rendering, heavy Python).
L8	Observability	OpenTelemetry + ClickHouse / Neon	Token billing, cost tracking, execution traces, immutable audit logs.

3

The Controlled Runtime Machine

NEW IN v1.2

To achieve strict determinism and eliminate LLM hallucinations inside production pipelines, the kernel enforces a strict **5-stage processing pipeline** for every computational step. Without this loop the system is probabilistic execution; with it, the system is a controlled runtime machine.



- 1

Plan (Suggestion Engine). The Planner Worker uses an LLM to evaluate task intent and breaks it into a highly structured Directed Acyclic Graph (DAG) or sequential steps. Output is structured JSON only — never free-form action.
- 2

Control (Gate / Policy Layer). Before any execution, the Capability Router validates input against the JSON Schema, checks security permissions, confirms the tool allowlist, and enforces SLA cost-guards. On failure it rejects the plan or triggers a rewrite.
- 3

Execute (Deterministic Muscle). The Executor Worker receives validated parameters and triggers the exact programmatic call (API tool or L7 heavy compute). Zero creativity, zero prompt alteration, zero model guessing — only `tool(input) → output`.
- 4

Verify (Verification Loop). Output is fed into specific validation contracts ensuring constraints are met (valid file sizes, structured schemas, token limits). On failure it triggers smart exponential-backoff retries or escalation flags.
- 5

Commit (State Engine). Once verified, state is finalized: asset URLs logged as *Artifacts*, reasoning/validation trails as *Evidence*, final outcomes as *Decisions*. A realtime signal is pushed to L1 via Supabase Broadcast.

3.1 Mapping the Pattern onto Source-A

Loop Stage	Source-A Component	Responsibility
Plan	Planner Worker	LLM generates a structured plan (DAG / steps).
Control	Capability Router + guards_config	Validation, cost check, permission, safety.
Execute	Executor Worker	Pure deterministic tool calling — no reasoning.
Verify	Evidence + Decisions + Validator	Post-execution checks and retry logic.
Commit	State Engine (Neon / Supabase)	Final commit + realtime notification.

Why this is the dividing line

Professional agentic systems (Cursor, Devin-like agents, agentic IDEs) bind plans to a code graph (AST / repo). Source-A's equivalent binding is the **Execution Contract Layer**: the plan is not just text, it is bound to a declared schema of allowed tools, expected output shapes, and validation rules. This is what turns “probabilistic agent soup” into a deterministic execution kernel driven by probabilistic planning.

Contracts & Data

Hierarchy, the Execution Contract Spec, and the locked schema

4 The Data Hierarchy & Execution Contracts

4.1 Data Hierarchy Structure

Every data entity exists in an explicit nested relational structure:

```
Tenant (Organization)
├── Project (Brand Workspace: Noetfield | Forge | TrustField | WitnessBC | 777)
│   ├── Factory (Specialized Pipeline: Board Review | Risk Scan | Ad Generator)
│   │   ├── Blueprint (Immutable, versioned runtime contract)
│   │   │   └── Run (Live execution instance)
│   │   │       └── Task (Atomic computation node)
│   │   │           ├── Artifact (Concrete output: video | audio | document file)
│   │   │           ├── Evidence (Audit trail: intermediate verification | proof)
│   │   │           └── Decision (System judgment: approved | rejected | escalated)
```

4.2 Blueprint Spec Example (Execution Contract Layer)

Factories are initiated strictly via configuration data contracts — never by writing custom worker scripts.

```

{
  "blueprint_name": "Forge_Ad_Generator",
  "version": 2,
  "inputs_schema": {
    "type": "object",
    "required": ["product_description", "target_audience"],
    "properties": {
      "product_description": { "type": "string", "maxLength": 1000 },
      "target_audience": { "type": "string" }
    }
  },
  "tools_config": {
    "allowed_tools": ["elevenlabs_tts", "modal_ffmpeg_render", "google_search"]
  },
  "guards_config": {
    "max_run_cost_usd": 0.15,
    "timeout_ms": 120000,
    "safety_filter": "strict"
  },
  "outputs_schema": {
    "type": "object",
    "required": ["final_video_url", "generation_audit_id"],
    "properties": {
      "final_video_url": { "type": "string", "format": "uri" },
      "generation_audit_id": { "type": "string", "format": "uuid" }
    }
  },
  "sla_default": "normal",
  "workflow_graph": [
    { "step": 1, "name": "Script_Writing", "capability": "writing", "sla": "normal" },
    { "step": 2, "name": "Voice_Generation", "capability": "tts", "sla": "draft" },
    { "step": 3, "name": "Video_Stitching", "capability": "video_render", "sla": "premium" }
  ]
}

```

5 Execution Contract Spec v1

NEW IN v1.2

A strict contract that converts **LLM output into safe, deterministic system execution**. Every capability/node must declare an execution contract before it is allowed to run.

5.1 Core Pipeline Definition

```

INPUT
↓
PLAN (LLM)
↓
CONTROL (Policy / Guard)
↓
EXECUTE (Tools)
↓
VERIFY (Validator)
↓
COMMIT (State Engine)

```

5.2 Execution Contract Object

Every Run carries exactly this structure:

```

{
  "run_id": "uuid",
  "project_id": "forge | noetfield | trustfield",
  "intent": "string",

  "plan": { "steps": [] },

  "context": {
    "user_input": "",
    "history": [],
    "constraints": {
      "budget": 0,
      "latency_ms": 0,
      "model_tier": "draft | normal | premium"
    }
  },

  "policy": {
    "allowed_tools": [],
    "blocked_tools": [],
    "data_access_level": "low | medium | high"
  },

  "execution": {
    "status": "pending | running | failed | success",
    "current_step": 0
  }
}

```

5.3 Plan Contract (LLM Output Schema)

The LLM is permitted to emit *only* this shape — nothing else:

```

{
  "steps": [
    {
      "step_id": 1,
      "type": "tool_call",
      "tool": "llm | elevenlabs | ffmpeg | supabase",
      "action": "string",
      "input": {},
      "expected_output": "artifact | text | file | json",
      "sla": "draft | normal | premium",
      "depends_on": []
    }
  ]
}

```

5.4 Control Layer Rules (Hard Gate)

Before any plan executes, all of the following must pass:

✓ Schema Validation

- `steps` must be an array
- each `tool` must exist in the registry
- no undefined actions

✓ Safety Rules

- no unauthorized external API calls
- no filesystem access outside the sandbox
- no recursive self-modifying plans

✓ Cost Check

- if `estimated_cost > budget` → reject or downgrade model tier

✓ Dependency Check

- the DAG must be acyclic
- no missing step references

5.5 Execution Layer Contract

The Executor accepts only this input, and applies no reasoning, planning, or model selection:

```
{ "tool": "string", "action": "string", "input": {} }  
  
// Rules: NO reasoning · NO planning · NO model selection · ONLY deterministic execution
```

5.6 Verification Layer

Every output is validated against the contract. Checks: schema validation, output completeness, artifact existence, integrity (hash / size / format), and policy compliance.

```
{ "status": "pass | fail", "reason": "string", "fix_required": true }
```

5.7 State Commit Layer

On verify PASS

- ✓ `runs.status = success`
- ✓ `tasks.status = success`
- ✓ artifacts saved
- ✓ decisions logged

On verify FAIL

- ✗ `runs.status = failed`
- ✗ retry policy triggered
- ✗ or rollback applied

5.8 Retry / Recovery Rules

- ✓ `max_retry = 3`
- ✓ exponential backoff with jitter
- ✓ fallback model if failure persists

5.9 Tool Registry (Critical)

All tools must be registered — never hardcoded into worker logic:

```
{  
  "elevenlabs": { "type": "audio", "capabilities": ["tts"] },  
  "ffmpeg":     { "type": "media", "capabilities": ["render", "merge"] }  
}
```

5.10 Cost + SLA Budgeting

Every step is tagged with an SLA tier that maps to a cost band:

SLA Tier	Model Class	Target Volume
draft	cheap model	~80%
normal	balanced model	~15%
premium	strong model	~5%

6

Production Database Schema (v1.2 — Locked Hybrid Model)

Supabase Engine

L1 / L2 frontend serving, user auth, and realtime Row-Level-Security events.

Neon Serverless Postgres

Primary engine for the heavy L3 / L4 / L8 workloads, using instant branch-cloning for safe testing of customer graphs.

6.1 Core Identity Layer

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE projects (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(100) UNIQUE NOT NULL,
  metadata    JSONB DEFAULT '{}'::jsonb,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE factories (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(100) NOT NULL,
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(project_id, slug)
);
```

6.2 Execution Layer (The Data Graph)

```
CREATE TABLE blueprints (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  factory_id  UUID NOT NULL REFERENCES factories(id) ON DELETE CASCADE,  
  version     INT NOT NULL DEFAULT 1,  
  inputs_schema JSONB NOT NULL DEFAULT '{}':jsonb,  
  tools_config JSONB NOT NULL DEFAULT '{}':jsonb,  
  guards_config JSONB NOT NULL DEFAULT '{}':jsonb,  
  outputs_schema JSONB NOT NULL DEFAULT '{}':jsonb,  
  workflow_graph JSONB NOT NULL DEFAULT '{}':jsonb,  
  is_latest   BOOLEAN DEFAULT TRUE,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE runs (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  blueprint_id UUID NOT NULL REFERENCES blueprints(id),  
  status      VARCHAR(50) NOT NULL DEFAULT 'pending',  
             -- pending | running | completed | failed | paused  
  current_node_id VARCHAR(100),  
  initiated_by  UUID,  
  created_at   TIMESTAMPTZ DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE tasks (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,  
  node_id     VARCHAR(100) NOT NULL,  
  capability_required VARCHAR(150) NOT NULL,  
             -- reasoning | tts | video_render | research | governance  
  status      VARCHAR(50) NOT NULL DEFAULT 'pending',  
             -- pending | queued | running | completed | failed  
  input_data  JSONB DEFAULT '{}':jsonb,  
  output_data JSONB DEFAULT '{}':jsonb,  
  idempotency_key VARCHAR(255) UNIQUE,  
  retry_count INT DEFAULT 0,  
  started_at  TIMESTAMPTZ,  
  completed_at TIMESTAMPTZ,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

6.3 Output Layer (Triple Segmentation)

```
CREATE TABLE artifacts (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,  
  task_id     UUID REFERENCES tasks(id) ON DELETE SET NULL,  
  artifact_type VARCHAR(50) NOT NULL, -- video_url | audio_url | raw_text | document  
  storage_path TEXT, -- Cloudflare R2 path  
  payload     JSONB DEFAULT '{}':jsonb,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE evidence (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,  
  task_id     UUID REFERENCES tasks(id) ON DELETE SET NULL,  
  source      VARCHAR(100) NOT NULL,  
  payload     JSONB NOT NULL,  
  confidence_score NUMERIC(4,3), -- 0.000 to 1.000  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE decisions (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,  
  task_id     UUID REFERENCES tasks(id) ON DELETE SET NULL,  
  decision_type VARCHAR(100) NOT NULL,  
  verdict     VARCHAR(50) NOT NULL, -- approved | rejected | escalated  
  rationale   TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

6.4 Control & Chaos-State Prevention

```
CREATE TABLE state_transitions (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,  
  from_state  VARCHAR(50) NOT NULL,  
  to_state    VARCHAR(50) NOT NULL,  
  transition_trigger VARCHAR(255) NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE run_logs (  
  id          BIGSERIAL PRIMARY KEY,  
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,  
  level       VARCHAR(10) NOT NULL, -- INFO | WARN | ERROR | CRITICAL  
  message     TEXT NOT NULL,  
  context     JSONB DEFAULT '{}':jsonb,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE task_logs (  
  id          BIGSERIAL PRIMARY KEY,  
  task_id     UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,  
  level       VARCHAR(10) NOT NULL,  
  message     TEXT NOT NULL,  
  context     JSONB DEFAULT '{}':jsonb,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

6.5 Realtime Telemetry & Cost Accounting

```
CREATE TABLE model_calls (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  task_id     UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,  
  provider    VARCHAR(100) NOT NULL, -- Google | DeepSeek | Anthropic  
  model_name  VARCHAR(100) NOT NULL,  
  capability_used VARCHAR(150) NOT NULL,  
  latency_ms  INT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE token_usage (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  model_call_id UUID NOT NULL REFERENCES model_calls(id) ON DELETE CASCADE,  
  prompt_tokens INT NOT NULL DEFAULT 0,  
  completion_tokens INT NOT NULL DEFAULT 0,  
  estimated_cost_usd NUMERIC(10,7) NOT NULL DEFAULT 0.0,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE execution_traces (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,  
  trace_id    VARCHAR(128) NOT NULL,  
  span_id     VARCHAR(64) NOT NULL,  
  parent_span_id VARCHAR(64),  
  operation_name VARCHAR(255) NOT NULL,  
  duration_ms INT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

6.6 Performance Optimization Indexes

```
CREATE INDEX idx_factories_project ON factories(project_id);  
CREATE INDEX idx_blueprints_factory ON blueprints(factory_id);  
CREATE INDEX idx_runs_blueprint ON runs(blueprint_id);  
CREATE INDEX idx_runs_status ON runs(status);  
CREATE INDEX idx_tasks_run ON tasks(run_id);  
CREATE INDEX idx_tasks_status ON tasks(status);  
CREATE INDEX idx_artifacts_run ON artifacts(run_id);  
CREATE INDEX idx_evidence_run ON evidence(run_id);  
CREATE INDEX idx_decisions_run ON decisions(run_id);  
CREATE INDEX idx_state_transitions_run ON state_transitions(run_id);  
CREATE INDEX idx_run_logs_run_level ON run_logs(run_id, level);  
CREATE INDEX idx_model_calls_task ON model_calls(task_id);  
CREATE INDEX idx_token_usage_call ON token_usage(model_call_id);  
CREATE INDEX idx_execution_traces_run ON execution_traces(run_id);
```

Runtime & Operations

Routing, workers, roadmap, and guarantees

7 Model Mapping Strategy (SLA-Driven Router)

The system abstracts individual base-model endpoints away from pipeline code. The Capability Router dynamically assigns tasks to backend providers based on SLA level and task nature. Models change; the Router does not.

80–85% · High Volume

Gemini 3.1 Flash-Lite
DeepSeek V4 Flash

Intake, validation, logging serialization, structured node inputs, quick evidence-tag assertions — executed at minimal API cost.

15–20% · Advanced Logic

Claude 3.5 / 4.6 Sonnet
Grok 4.1 Fast

Governance evaluations, intricate Forge code generation, multi-layered planning, and final regulatory audit approvals.

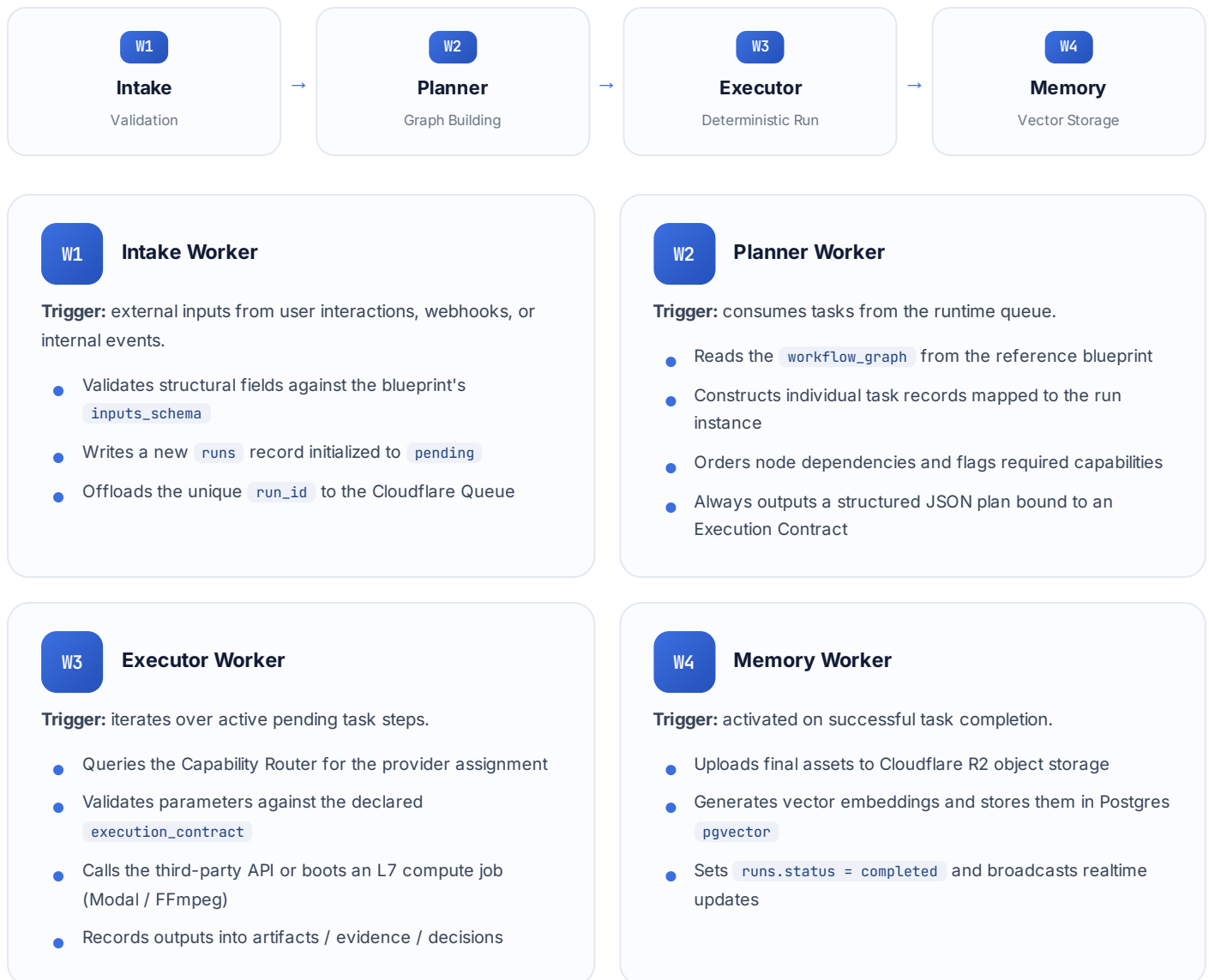
Capability Class	Requested SLA	Target LLM Mapping (Default)	Volume
research / drafting	Draft	Gemini 3.1 Flash-Lite / DeepSeek V4 Flash	~70%
reasoning / analysis	Normal	DeepSeek Reasoner / Gemini 1.5 Pro	~20%
writing / verification	Premium	Claude 3.5 / 4.6 Sonnet	~8%
governance / audit	Critical	Claude 3.5 / 4.6 Sonnet (Anthropic)	~2%

7.1 Capability Router — Core Logic

```
// Tool Registry lives in the database, never hardcoded
const CAPABILITY_MAP = {
  "intake":    { providers: ["gemini-flash-lite"],    fallback: "deepseek-flash" },
  "research":  { providers: ["deepseek-v4-flash"],  fallback: "gemini-pro"     },
  "reasoning": { providers: ["deepseek-reasoner"],  fallback: "claude-sonnet"  },
  "writing":   { providers: ["claude-sonnet"],       fallback: "gemini-pro"     },
  "governance": { providers: ["claude-sonnet"],      fallback: null /* escalate */ },
  "tts":       { providers: ["elevenlabs"],         fallback: "openai-tts"    },
  "video_render": { providers: ["modal-ffmpeg"],    fallback: null /* queue */ },
};

async function resolve({ capability, sla }) {
  const config = await db.getCapabilityConfig(capability); // from Tool Registry
  const provider = selectProvider(config, sla);
  if (circuitBreaker.isOpen(provider)) {
    return config.fallback ?? escalate(capability);
  }
  return provider;
}
```

Source-A eliminates per-product microservices. The entire multi-product runtime functions using exactly four immutable workers.



9.1 Phased Implementation Milestones

Phase	Focus	Deliverables
Phase 1	Core Kernel current	Identity setup, Supabase ↔ Neon database structure, versioned blueprint graphs, initial run-state verification.
Phase 2	Execution Engine	Cloudflare Queue pathways, the 4 core workers, and output/evidence logging tables.
Phase 3	Router Intelligence	Capability translation rules, tool-registry mappings, and contract evaluation.
Phase 4	Full Observability	OpenTelemetry hooks, token-expenditure tracking, and execution-trace graphs.

9.2 Operational Cost Comparison

Operational Metric	Traditional Infrastructure	Source-A Unified OS
Fixed cost overhead	\$500-\$1,500/mo (idle server maintenance)	\$55-\$190/mo (pure serverless, usage-based)
New product onboarding	Provision servers, siloed apps, unique databases	A single row in the <code>projects</code> table
Compute scaling	Individual specialized worker nodes per feature	4 core workers driven by JSON workflow graphs
Audit trails & evidence	Manually written tracking architectures	Automated, native recording of evidence outputs

Once the Execution Contract is enforced end-to-end, the kernel provides hard guarantees:

 Deterministic execution not probabilistic	 Full traceability audit-ready by default	 Replayable runs reconstruct any execution	 Tool-level isolation one failure ≠ whole run	 Model-agnostic swap providers without code change
--	---	--	---	--

✓ Locked & Agreed

- **Source-A OS identity:** formally an immutable AI Operating System, not a cron-automation wrapper.
- **No code logic inside workers:** workers are deterministic machinery; product variation lives in the Data Graph.
- **5-Stage Control Loop:** Plan → Control → Execute → Verify → Commit is mandatory for every step.
- **Execution Contracts:** every capability/node declares a contract before it may run.
- **Observability mandatory:** realtime cost, token tracking, and trace auditing active from day one.
- **Hybrid persistence:** Neon for high-write state; Supabase for auth, RLS, and realtime.

FINAL SYSTEM DEFINITION

Source-A is not an AI system.

It is a deterministic execution kernel driven by probabilistic planning.

SOURCE-A KERNEL v1.2 — LOCKED & APPROVED

This document is the official SSOT for the Source-A project. No architectural decision may be made without reference to it.

Noetfield Systems Inc. | TrustField Technologies Inc.

Vancouver, BC — 2026 · Target Ledger Deployment: v1.2 Production